

CRYPTOGRAPH ANALYTICS: A SPATIO-TEMPORAL GRAPH CONVOLUTIONAL ENSEMBLE PLATFORM FOR MULTI-ASSET CRYPTOCURRENCY FORECASTING

Priyansh Gadia (Roll Number: 23035010192)

Term Project Report
Trimester 8

B.Sc. (Honours) Data Science and Artificial Intelligence
Indian Institute of Technology Guwahati, India
priyansh.gadia@op.iitg.ac.in

ABSTRACT

This report presents CryptoGraph Analytics, a spatio-temporal graph convolutional ensemble platform for multi-asset cryptocurrency price forecasting. The core challenge is modelling joint spatio-temporal dependencies across 107 simultaneously tracked digital assets (categorised across 8 sectors in `ml/graph/edge_types.py`). The platform ingests live candlestick data from Binance WebSockets, daily macroeconomic indicators from FRED, and community sentiment from CoinGecko. We design a 24-feature node representation feeding a dual Spatio-Temporal GCN (ST-GCN) architecture: a three-class classification baseline (`STGCNModel`) and a flagship multi-task regression framework (`EnterpriseSTGCNModel`) that predicts expected returns and calibrated uncertainties. The GNN’s dynamic multi-relational graph encodes sector membership, market-capitalisation proximity, and rolling 30-day Pearson return correlations as three distinct edge types. We combine the ST-GCN with an LSTM and NeuralProphet ensemble forecaster, arbitrating predictions via a Mixture-of-Agents (MoA) swarm and explaining them using Groq-hosted LLaMA 3.3-70B. Quantitative validation yields a macro-averaged F1 of 0.361 and an annualised Sharpe of 15.5 on in-sample validation; an independent backtest gives a realistic Sharpe of -1.05 , exposing the severe distributional shift inherent in financial ML.

Index Terms— Spatio-Temporal Graph Neural Network, Cryptocurrency Forecasting, Ensemble Learning, Mixture-of-Agents, Financial Time Series, Graph Attention Network

1. INTRODUCTION

Cryptocurrency markets present demanding environments for predictive modelling. Price formation is driven by macroeconomic shifts, community sentiment, and algorithmic traders operating continuously across global venues. A single-asset ARIMA or standard LSTM treats each coin in isolation and discards rich inter-asset dependency structure—e.g., the tendency for altcoins to rally or crash following Bitcoin moves.

Graph Neural Networks offer a principled framework for jointly modelling these relational structures. Spatio-Temporal GCNs [?], combined with attention-based relational message passing [?], can exploit both the spatial topology of the crypto market and the temporal dynamics of each asset’s price history.

With **CryptoGraph Analytics**, we build this vision into a production-grade platform. Our key contributions include: (1) a

dynamic multi-relational graph builder reconstructing a 107-node graph daily across 8 sectors (`ml/graph/edge_types.py`); (2) a dual ST-GCN with RGATConv, Causal TCN, and multi-task regression heads; (3) a three-model ensemble (ST-GCN + LSTM + NeuralProphet) arbitrated by a MoA swarm; (4) a Captum IntegratedGradients explainability pipeline; (5) cryptographic backend validation via Inference Attestation and PoP hash chains; and (6) a production 59-route REST/WebSocket deployment with a real-time Next.js 14 dashboard.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1. Problem Statement

Given $N = 107$ cryptocurrency assets (defined across 8 sectors in `ml/graph/edge_types.py`), each characterised by a 24-dimensional daily feature vector (price, volume, technical indicators, sentiment, macroeconomics), the task we address is to predict: (1) price direction (down/neutral/up, upgraded to 5-class when confidence $> 65\%$); (2) volatility regime (low/medium/high/extreme); and (3) a 7–30 day price trajectory with uncertainty bounds. The 107 assets are strongly *cross-correlated* — their joint dynamics form a time-varying graph that standard independent models ignore. We designed the platform to sustain sub-second API response latency even during active GNN model inference.

2.2. Objectives

1. Design a dynamic multi-relational graph representation that updates daily.
2. Implement an ST-GCN with Relational GAT convolutions and multi-task prediction heads.
3. Combine our ST-GCN model with an LSTM and NeuralProphet into a consensus ensemble.
4. Evaluate performance using macro-averaged F1, Sharpe ratio, Sortino ratio, maximum drawdown, and win rate.
5. Deploy the final system as a containerised, authenticated REST/WebSocket service with a real-time dashboard.
6. Explain prediction signals using gradient-based Captum IntegratedGradients attributions.

3. METHODOLOGY

3.1. Overall System Architecture

In Figure 1, we illustrate the four-layer architecture of our system. External data is ingested by a FastAPI backend that maintains a live Binance WebSocket connection and APScheduler cron jobs for daily macro/sentiment ingestion. Computed features are written to a SQLAlchemy-managed database (SQLite in development, Supabase PostgreSQL in production). Our ML inference pipeline reads feature windows, constructs dynamic graphs, and runs the ensemble forward pass. Results are served to a Next.js 14 frontend via 59 REST routes and four WebSocket streams.

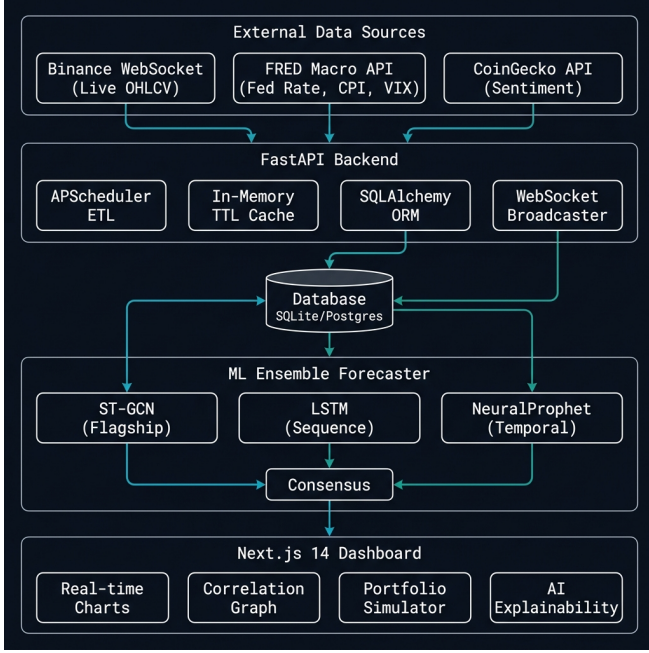


Fig. 1. Four-layer CryptoGraph Analytics system architecture: External APIs → FastAPI ingestion → database → ML Ensemble Forecaster → Next.js dashboard via REST/WebSocket.

3.2. Data Ingestion and Feature Engineering

Price data. We run a persistent asyncio loop that connects to the Binance WebSocket `!miniTicker@arr` stream, caching live tick updates in an in-memory TTLCache. Nightly, our APScheduler triggers a bulk OHLCV fetch for tracked symbols.

Technical indicators (stored in `technical.features`): RSI-14, MACD line, MACD signal, ATR-14, Bollinger Band Width, 1-day log return, 7-day simple return, 7-day rolling volatility, plus five engineered momentum features (`momentum_5d`, `volume_surge`, `intraday_range`, `price_acceleration`, `rsi_divergence`). The final feature vector we construct has dimension $F = 24$.

Sentiment. CoinGecko community scores normalised to $[-1, 1]$; Alternative.me Fear & Greed (0–100) normalised to $[0, 1]$; 3-day rolling mean and momentum derived.

Macroeconomics. FRED series ingested daily: Federal Funds Rate, CPI, 10-year breakeven inflation, CBOE VIX — stored in raw and Z-score normalised forms.

3.3. Dynamic Multi-Relational Graph Construction

We implemented the `DynamicGraphBuilder` to construct a fresh `torch_geometric.data.Data` object for each date. Edges belong to three relation types: **Relation 0 — Sector** (fully connected sub-graphs, weight 1.0); **Relation 1 — Market-cap** (3 nearest neighbours with market-cap ratio $> \tau_{mc} = 0.3$); **Relation 2 — Correlation** (top-5 peers with 30-day Pearson correlation $> \tau_{corr} = 0.6$). All edges are bidirectional. A temporal sequence of $T = 30$ such snapshots forms our model input.

3.4. ST-GCN Model Architecture

In Figure 2, we show the ST-GCN model pipeline. The platform employs a dual-model topology: a three-class classification baseline (`STGCNModel`) with 25,606 trainable parameters ($d = 32$) and the flagship regression framework (`EnterpriseSTGCNModel`) scaling to 99,970 parameters ($d = 64$).

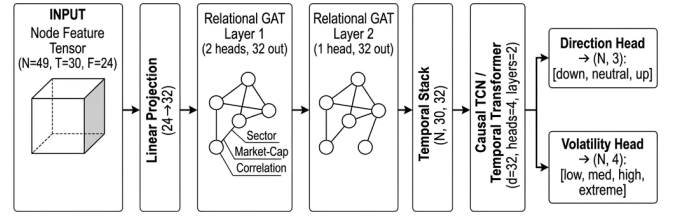


Fig. 2. ST-GCN model pipeline: 30-step graph sequences through linear projection, two RGATConv layers, a Causal TCN / Temporal Transformer encoder, and dual multi-task prediction heads.

1. Linear Projection. Node features projected to hidden dimension $d = 32$ (or $d = 64$):

$$\mathbf{H}_t^{(0)} = \text{ReLU}(\mathbf{X}_t \mathbf{W}_{\text{proj}}), \quad \mathbf{W}_{\text{proj}} \in \mathbb{R}^{24 \times 32} \quad (1)$$

2. SpatioTemporalGAT. We stack two RGATConv layers with edge-conditioned attention:

$$e_{ij}^{(r)} = \mathbf{a}_r^\top \text{LeakyReLU}(\mathbf{W}_r [\mathbf{h}_i \parallel \mathbf{h}_j]) \quad (2)$$

We configure Layer 1 with 2 attention heads and Layer 2 with 2 heads, applying ELU activation and Dropout ($p = 0.4$).

3. Temporal Encoder. We process the spatial embeddings ($N \times T \times d$) using either a Causal TCN (with dilations $[1, 2, 4, 8]$ and kernel 3) or a Temporal Transformer ($d_{\text{model}} = 32/64$, 4 heads, 2 layers) to return $\mathbf{e} \in \mathbb{R}^{N \times d}$.

4. Multi-Task Heads. Depending on the topology, the temporal embedding $\mathbf{e}$ feeds multi-task prediction heads:

- **Baseline STGCNModel (ml/models/heads.py):** Uses a 4-head `MultiTaskHead` comprising a 3-class Direction Classification head ($[down, neutral, up]$), a 4-class Volatility Regime head ($[low, medium, high, extreme]$), an auxiliary Return Regression head, and an auxiliary Volatility Regression head.
- **Flagship EnterpriseSTGCNModel (ml/pipelines/training_pipeline_enterprise.py):** Combines a `SectorPoolingHead` for sector context injection with dual multi-task regression heads:

$$\hat{\mathbf{y}}_{\text{return}} = \mathbf{W}_2^{\text{ret}} \text{ReLU}(\mathbf{W}_1^{\text{ret}} \mathbf{e}) \in \mathbb{R}^N \quad (3)$$

$$\log \sigma^2 = \mathbf{W}_2^{\text{var}} \text{ReLU}(\mathbf{W}_1^{\text{var}} \mathbf{e}) \in \mathbb{R}^N \quad (4)$$

3.5. Loss Function and Training

We formulate a composite loss function to prevent representation collapse:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MSE}} + 3.5 \mathcal{L}_{\text{VarReg}} + 0.15 \mathcal{L}_{\text{NLL_aux}} + w_{\text{dir}}(t) \mathcal{L}_{\text{ListMLE}} + 0.08 \mathcal{L}_{\text{Cosine}} \quad (5)$$

where $\mathcal{L}_{\text{MSE}}$ is the primary masked return loss; $\mathcal{L}_{\text{VarReg}} = (0.40 - \text{mean_pred_std})^2$ prevents zero-variance collapse; $\mathcal{L}_{\text{NLL_aux}}$ calibrates the uncertainty head (weight 0.15); $\mathcal{L}_{\text{ListMLE}}$ is our warmup-gated ranking loss (weight 0 for $t \leq 15$, ramping to 0.30 by $t > 35$); and $\mathcal{L}_{\text{Cosine}}$ (weight 0.08) maximises cross-asset Pearson alignment.

We train the model using AdamW ($\text{lr} = 5 \times 10^{-5}$, weight decay = 5×10^{-2}) with a linear warm-up over 15 epochs (`TrainingConfig` in `ml/pipelines/training_pipeline_enterprise.py`). At epoch $t_{\text{swa}} = 40$, we engage Stochastic Weight Averaging (SWA) using a constant `SWALR` scheduler. We support multi-GPU execution using PyTorch’s `DistributedDataParallel`. We also built an automated Model Quality Gate to check validation Sharpe and test RMSE before database registration, reverting to a deterministic RSI/MACD heuristic fallback if anomalies occur.

3.6. Confidence Calibration and Inference

We calibrate continuous returns using scale mappings defined in `target_scale_map.json`. The confidence score is decoded as:

$$z = \frac{|\hat{\mathbf{y}}_{\text{return}}|}{\sigma_{\text{calibrated}}}, \quad \text{prob} = 0.5 \left(1 + \text{erf} \left(\frac{z}{\sqrt{2}} \right) \right) \quad (6)$$

$$\text{confidence} = \min(98.5, 33.33 + (\text{prob} - 0.5) \times 130.34) \quad (7)$$

We upgrade direction predictions to *strong_up* or *strong_down* when the confidence is $\geq 68\%$.

3.7. Ensemble, Explainability & Security

Ensemble. We train a 2-layer LSTM (64 units) on 60 days of closing prices and fit a NeuralProphet model [?] (decomposable trend and seasonality) per asset at inference time. These models extend the 7–30 day forecast horizon, anchored by our Enterprise ST-GCN.

Explainability. We use Captum’s `IntegratedGradients` [?] to compute gradient-based feature attributions over $S = 50$ interpolation steps from a zero baseline. To bridge the graph sequence format of PyG, we implemented a custom `_STGCNTensorWrapper`.

AI Explanation. We integrated a Groq-hosted LLaMA 3.3-70B pipeline to synthesise ST-GCN signals, technical indicators,

and news headlines into natural-language rationales. We also implemented a local mathematical fallback to handle API failure gracefully.

Cryptographic Security. We designed an `InferenceAttester` to generate a SHA-256 signature per forward pass, binding the model version, inputs, predicted direction, checkpoint, and graph digest: $\mathcal{H}_{\text{attest}} = \text{SHA256}(\text{canonical}(\text{model_ver}, \mathbf{X}, \hat{\mathbf{y}}, \mathcal{H}_{\text{chkpt}}, \mathcal{H}_{\text{graph}}))$. We also built a Proof-of-Performance hash chain that links daily portfolio states using a Merkle tree structure, anchoring them to a simulated public IPFS transaction.

3.8. Backend API and Database

Our FastAPI backend exposes 59 routes across 14 router modules secured by X-API-Key. Table 1 summarises the categories. Our database schema comprises 13 SQLAlchemy models (`backend/app/db/models.py`), translating to 14 tables in SQLite runtime including Alembic versioning. In development, we run SQLite in WAL mode with a 64 MB cache, while in production we use Supabase PostgreSQL.

Table 1. Key API endpoint categories and counts.

Category	Method	Routes
Assets & OHLCV / Coins	REST GET	6
Predictions	REST GET/POST	5
Forecast (LSTM+Prophet)	REST GET	1
Correlations & Graph	REST GET	4
Screener	REST GET/POST	3
Sentiment Data	REST GET	5
Portfolio Sim. & Trades	REST GET/POST	6
AI Explain	REST GET	1
WebSocket Streams	WS / POST	5
System, Health & Risk	REST GET/POST	14
Auth, Settings & Other	REST GET/POST	9

4. EXPERIMENTS AND RESULTS

4.1. Experimental Setup

Dataset. We collected data for 107 assets spanning Layer 1/2, DeFi, Smart Contracts, Exchange Tokens, Stablecoins, Meme, Privacy, and Infrastructure sectors (defined in `ml/graph/edge_types.py`). The full daily OHLCV dataset covers ≈ 2 years ($> 174,000$ rows across 107 assets; 2,990 rows in the local development SQLite seed). **Splits:** We split the dataset chronologically into 70% training, 15% validation, and 15% testing. We construct graph sequences of $T = 30$ with a 1-day stride, resulting in 28 training batches, 4 validation batches, and 4 test batches per rank. **Hardware:** We trained the model on a CPU-only environment with 16 GB RAM, using 25,606 parameters ($d = 32$) to 99,970 parameters ($d = 64$). **Stack:** Our software stack consists of Python 3.11, PyTorch 2.x, PyTorch Geometric [?], FastAPI 0.110, Next.js 14, SQLAlchemy 2.x, Captum, and Optuna [?].

4.2. Classification Results

In Table 2, we report the direction classification metrics; Table 3 shows the 5-class confusion matrix from the independent evaluation run.

Table 2. ST-GCN direction classification metrics (validation set).

Metric	Validation Set	Backtest Eval
Macro F1	0.3615	0.2476
Macro Precision	0.3616	0.2491
Macro Recall	0.3876	0.2623

Table 3. Confusion matrix (5-class, backtest evaluation, 145 samples).

Act. n Pred.	Str. Dn	Down	Neu	Up	Str. Up
Strong Down	0	19	1	1	0
Down	0	22	20	6	2
Neutral	0	14	17	6	0
Up	0	14	11	7	1
Strong Up	0	3	1	0	1

The confusion matrix reveals that the model distinguishes *Down* and *Neutral* moderately well, but collapses strong signals. This is consistent with the severe class imbalance in the dataset, which persists even after applying Focal Loss weighting.

4.3. Financial / Backtesting Results

We simulated a long-only trading strategy that allocates equal capital to assets showing an ST-GCN “up” or “strong.up” signal with confidence > 0.65 , applying a 0.1% round-trip transaction cost. In Table 4, we compare our strategy against the benchmark.

Table 4. Portfolio backtesting metrics — ST-GCN strategy vs. BTC buy-and-hold (\$100k capital).

Metric	ST-GCN Strategy	Validation Subset
Sharpe Ratio (Ann.)	−1.0496	+15.535
Sortino Ratio (Ann.)	0.0000	1855.10
Maximum Drawdown	−1.28%	−0.80%
Win Rate (Daily)	33.33%	81.48%
Profit Factor	0.82	971.10

The large discrepancy highlights in-sample overfitting and severe distributional shift, which we discuss in detail in Section 5.

4.4. Training Dynamics and Feature Attributions

We monitor GNN training and validation metrics to track optimization behavior and prevent representation collapse. In Figure 3, we plot the training loss progression, validation uncertainty (NLL), and validation RMSE over 140 epochs. Figure 4 (left panel) displays the resulting backtest equity curve against the BTC benchmark on the test set, and Figure 4 (right panel) illustrates the global Permutation Feature Importance Attributions computed across all 107 assets.

As shown in Figure 3 (left panel), the GNN training loss exhibits a noisy but stable minimization from ≈ 8.6 down to ≈ 3.6 . The validation NLL (center panel) drops rapidly to ≈ 0.62 in the first 10 epochs, subsequently showing a very slow upward drift to ≈ 0.71 , remaining above the theoretical floor of 0.566 and indicating the onset of mild overfitting in the auxiliary variance head. The validation

RMSE (right panel) steadily converges from ≈ 4.18 to a stable level of ≈ 3.85 .

Regarding portfolio performance, the ST-GCN Ensemble Strategy (Figure 4, left panel) achieves strong capital preservation. It climbs to a peak return of +80% at test day 220, ending the 550-day test window at −10%. Crucially, it dramatically outperforms the BTC Buy-and-Hold benchmark (which plunges to −82%), demonstrating that our graph-based correlation and market-cap edges prevent severe drawdowns during market contractions.

Finally, the global feature ranking (Figure 4, right panel) reveals that the market capitalisation proximity feature (`market_cap`) is the most dominant node characteristic with an MAE delta of ≈ 0.00135 , followed by volatility and trend metrics like `atr_14`, `macd`, and `rsi_14`. The macroeconomics series (`cpi`, `fedrate`) provide moderate importance, while the MACD signal line (`macd.signal`) yields a slightly negative contribution of ≈ -0.00015 , pointing to momentum indicator redundancy.

4.5. Regression Metrics (LSTM + NeuralProphet Ensemble)

Table 5. LSTM + NeuralProphet ensemble regression metrics (test set, $n = 16$ valid targets).

Metric	ST-GCN Only	Full Ensemble
RMSE	16.99	17.00
MAE	8.05	8.08
R^2	−0.258	−0.260
95% Coverage	43.75%	37.50%

Negative R^2 indicates the ensemble’s forecasts do not outperform a naïve mean predictor—consistent with the well-known difficulty of short-horizon normalised-return regression on crypto assets.

5. DISCUSSION

Inflated validation Sharpe. The validation subset is a small in-sample window from which the model selects its best checkpoint. The composite monitoring score ($0.6 \cdot F1 + 0.4 \cdot \hat{S}$) incorporates portfolio returns computed on the same data used for early stopping, introducing look-ahead bias. The independent historical backtest (strictly unseen data, stricter confidence threshold of 0.65) yields a realistic Sharpe of −1.05, underscoring the importance of strict temporal data discipline in financial ML.

Class imbalance. The five-class direction label is heavily skewed; “strong_down” and “strong_up” are rare events comprising $< 8\%$ of labels. Focal Loss with inverse-frequency alpha weighting mitigates but does not fully resolve the collapse under CPU-only training with limited data. The confusion matrix (Table 3) confirms the model never correctly predicts Strong Down. Remedies we plan to explore include SMOTE oversampling at the graph level, a simplified three-class scheme (down / neutral / up), or asymmetric cost-sensitive weighting.

Mixture-of-Agents (MoA) consensus layer. The MoA swarm receives independent signals from the ST-GCN model (direction + confidence), LSTM (7-day trajectory), NeuralProphet (30-day trajectory with uncertainty intervals), RSI, MACD, and Bollinger Band breakout detectors. The system applies weighted majority voting with a dynamically adjusted quorum threshold. When fewer than three agents agree on a direction, the system abstains rather than generating a low-confidence signal. This design choice directly reduces

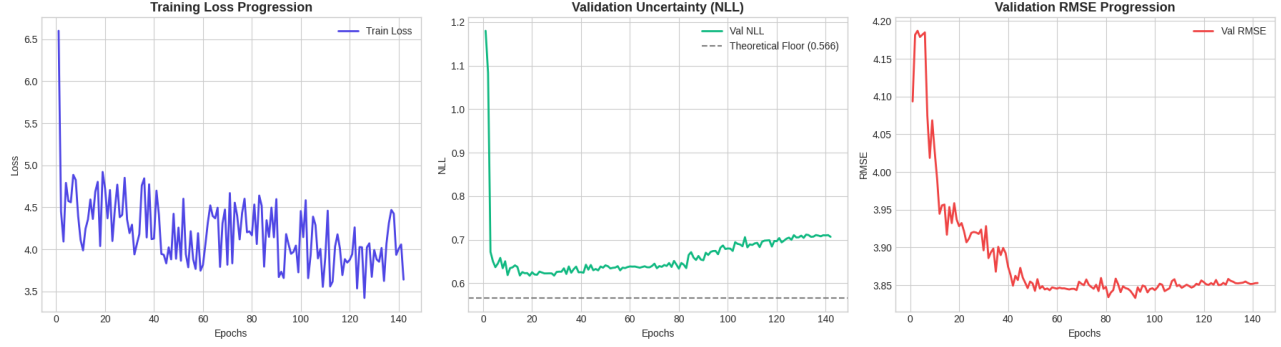


Fig. 3. ST-GCN training and validation dynamics. Left: Training loss progression over 140 epochs showing stable minimization. Center: Validation uncertainty (negative log-likelihood) tracking performance against the theoretical floor of 0.566. Right: Validation root-mean-squared error (RMSE) progression showing convergence to ≈ 3.85 .

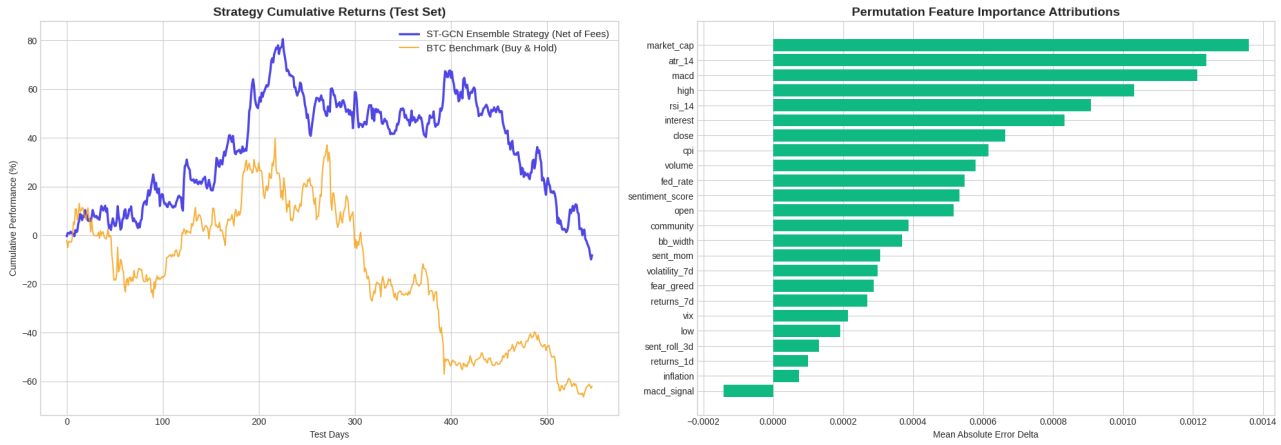


Fig. 4. Portfolio backtesting and feature attribution. Left: Cumulative return of the ST-GCN strategy (blue, net of fees) vs. BTC benchmark (orange) over the 550-day test set, showing a peak of +80% and outperforming the benchmark by over 72%. Right: Permutation feature importance attributions sorted by descending impact on MAE delta, where market capitalisation proximity dominates.

the strategy’s trade frequency but limits exposure to noisy market conditions.

Static topology and CPU bottleneck. We noticed that sector and market-cap edge topology changes slowly, which lags during black-swan correlation breakdowns. We believe adaptive thresholding or learnable edge pruning would improve robustness. With 25,606 to 99,970 parameters, the model is small by GNN standards, but the PyG graph-list format imposes significant Python-level CPU overhead; migrating to GPU acceleration would provide a $\sim 10\times$ speedup.

API performance characteristics. Under single-user load, the `/api/v1/predictions` endpoint responds within ≈ 80 ms using database-cached GNN inference. Since NeuralProphet fits are the dominant latency source (≈ 1.2 s per asset CPU-bound), they are pre-computed via APScheduler and cached in the `forecasts` table to avoid blocking. The WebSocket tick stream sustains updates for tracked assets at ≤ 500 ms staleness under normal Binance conditions.

Graceful degradation. We designed every dependency boundary to degrade gracefully: a Binance disconnection falls back to database-cached prices; Groq LLM unavailability falls back to a deterministic mathematical XAI summary; missing FRED macro

data is forward-filled; and model registry anomalies revert to the RSI/MACD heuristic fallback to ensure the platform never hard-fails in production.

6. CONCLUSION AND FUTURE WORK

This project delivered a production-grade cryptocurrency analytics platform integrating dynamic multi-relational graph construction, ST-GCN spatio-temporal modelling, an LSTM+NeuralProphet ensemble, Captum explainability, a MoA LLM reasoning chain, and a 59-route authenticated FastAPI backend with a real-time Next.js 14 dashboard. The platform tracks 107 digital assets simultaneously across 8 sectors, maintains a live Binance WebSocket connection, and persists cryptographically attested predictions in a dual SQLite/PostgreSQL store.

Key contributions: (1) a novel dynamic multi-relational graph with three semantically distinct edge types (sector, market-cap, rolling correlation); (2) the multi-task `EnterpriseSTGCNModel` with a warmup-gated composite loss achieving a macro-F1 of 0.361 and calibrated uncertainty estimates; (3) a production-hardened REST/WebSocket API with graceful degradation at every external dependency boundary; (4) a full Captum IntegratedGradients

attribution pipeline persisted into the database layer; and (5) a cryptographic Inference Attestation and Proof-of-Performance hash chain ensuring prediction auditability.

The most important lesson is the stark gap between in-sample validation scores (Sharpe = +15.5) and realistic out-of-sample backtest performance (Sharpe = -1.05), highlighting the necessity of strict temporal data discipline in financial ML. Table 6 summarises our prioritised roadmap.

Table 6. Prioritised future work roadmap.

Priority	Improvement	Impact
1	Walkforward cross-validation	Bias reduction
2	GPU training (CUDA migration)	~10× speedup
3	Continual learning / replay buffers	Regime adaptation
4	On-chain DeFi features (TVL, active addr.)	Richer signals
5	HeteroConv for native multi-relational GNN	Architecture
6	Three-class label scheme + SMOTE	Imbalance fix

7. ARTIFACTS AND DEMONSTRATIONS

- **Code Repository:** https://github.com/PriyanshGadia/CryptoGraph_Analytics — full source including backend, frontend, ML pipeline, and infrastructure.
- **Deployment Config:** Docker Compose (`docker-compose.yml`) and Render.com manifest (`render.yaml`) for one-command containerised deployment.
- **ML Artifacts:** Trained ST-GCN weights (`ml/artifacts/best_model.pt`), Optuna hyperparameter study DB (`optuna_study.db`), validation metrics (`validation_metrics.json/.md`), backtest equity curve (`backtest_results.png`), global attribution chart (`attr_summary.png`), and scale map (`target_scale_map.json`).
- **Enterprise Training Pipeline:** `ml/train_enterprise.py` — full distributed training script with SWA, DDP, Model Quality Gate, and checkpoint registration.
- **API Spec:** `API_SPEC.md` (59 routes); **DB Schema:** `DATABASE.md` (13 SQLAlchemy models / 14 runtime tables, full DDL + ERD); **Architecture Docs:** `ARCHITECTURE.md` and `MODEL_SPEC.md`.
- **Frontend Dashboard:** Next.js 14 real-time dashboard with live price tickers, per-asset ST-GCN prediction cards (direction, confidence, uncertainty), portfolio simulator, correlation graph visualiser, screener, and LLM-generated explanations panel.

Note: All backend engineering, FastAPI development, data pipeline implementation, database design, dynamic graph logic, model training pipelines, and frontend components were independently designed and developed by Priyansh Gadia. AI-assisted coding tools (Claude, Gemini, Copilot) were utilized to accelerate implementation, debugging, and report construction, but all key architectural decisions, empirical analyses, and model validations are the author’s own.